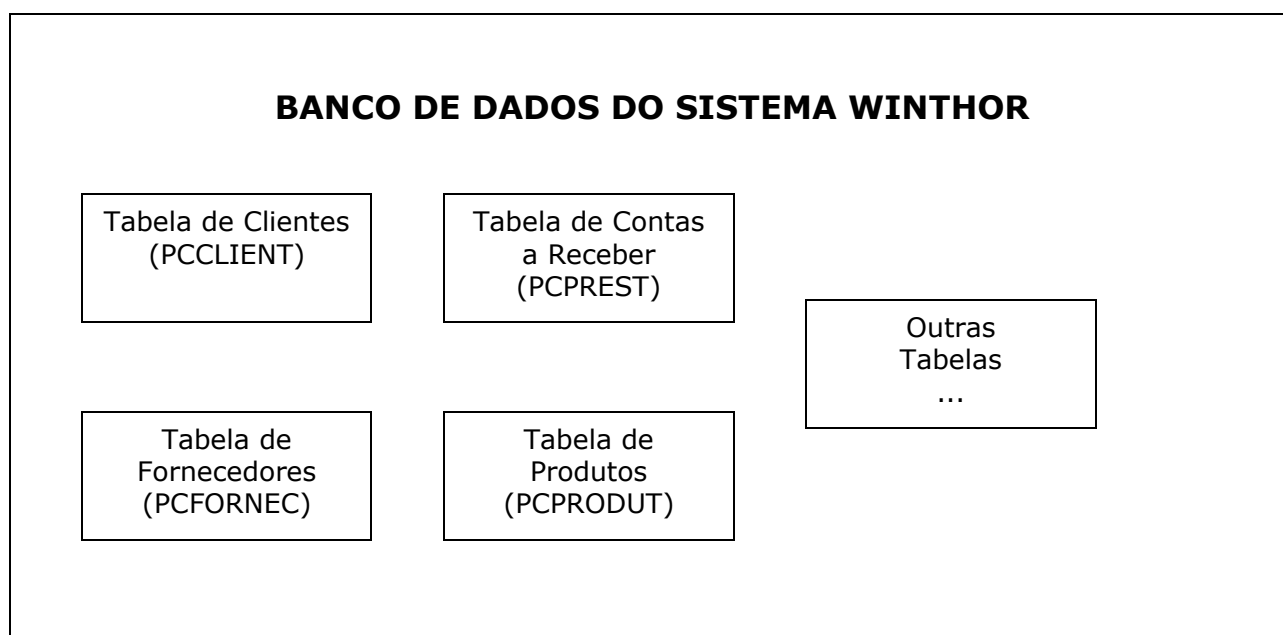

1 Banco de Dados

1.1 O que é um Banco de Dados (BD)

Bancos de dados é uma estrutura onde dados são armazenados de maneira organizada. Essas estruturas costumam ter a forma de **tabelas**. Podemos dizer que Banco de Dados é um conjunto de Tabelas.

Um **banco de dados** é um conjunto de dados com uma estrutura regular. Um banco de dados é normalmente, mas não necessariamente, armazenado em algum formato de máquina lido pelo computador.

O Banco de Dados do sistema Winthor possui mais de uma centena de tabelas. Dentre elas, podemos citar algumas tabelas muito utilizadas, como, por exemplo, a Tabela de Clientes (PCCLIENT), Tabela de Produtos (PCPRODUT), Tabela de Fornecedores (PCFORNEC) e Tabela de Contas a Receber (PCPRPEST), entre outras.



1.2 Conceitos

1.2.1 Tabelas

As **tabelas** consistem na estrutura básica para armazenar dados. Uma tabela é composta de campos que possuem duas qualidades que determinam os dados que podem ser armazenados neles. A primeira é a qualidade do **tipo de dado**, que pode ser *texto, memorando, número, moeda, data/hora, sim/não, contador*. A segunda propriedade refere-se à **lista de propriedades** relacionada a cada campo, como *tamanho, formato, casas decimais, legenda, valor padrão, regra e texto de validação*.

EXEMPLO DE ESTRUTURA DE TABELA

NOME	NULL	TYPE
CODPROD	NOT NULL	NUMBER (6,0)
DESCRICAO	NOT NULL	VARCHAR2 (40)
EMBALAGEM	NOT NULL	VARCHAR (12)
UNIDADE		VARCHAR2 (2)
CODEPTO	NOT NULL	NUMBER (6,0)
DTCADASTRO		DATE

1.2.2 Registro X Campo

As tabelas são formadas por uma ou mais **colunas**, zero ou mais **linhas**. Usamos uma tabela para armazenar dados sobre uma ou mais ocorrências de uma determinada entidade de nosso Banco de Dados.

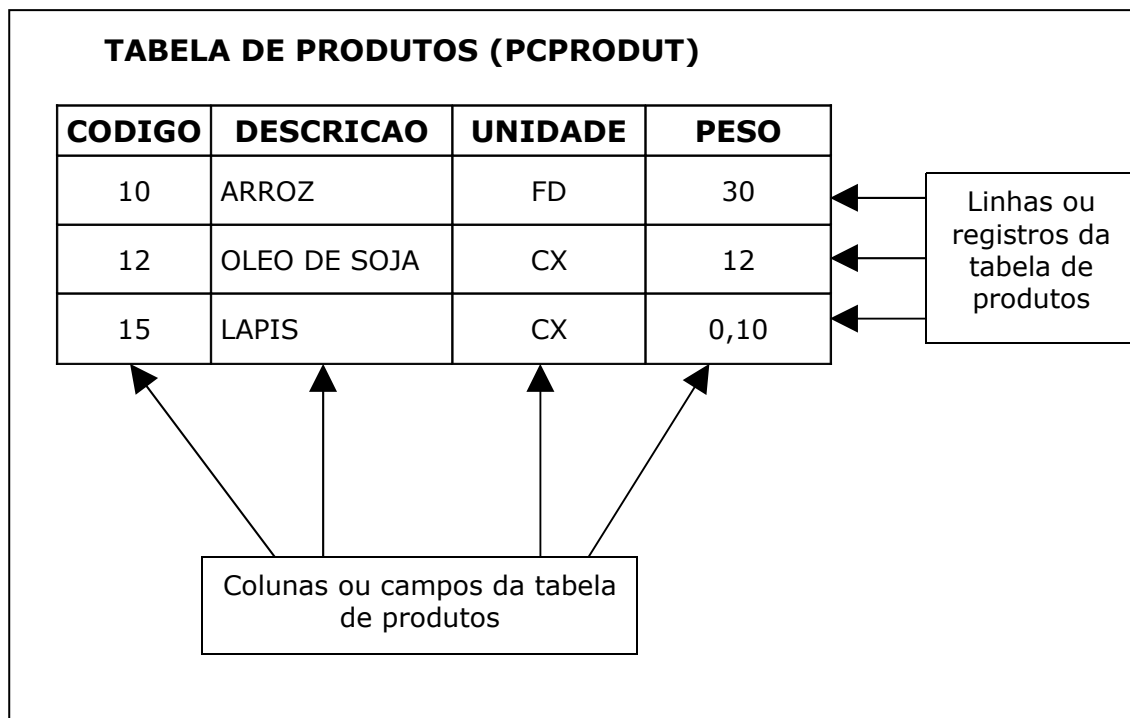
A cada coluna da tabela damos o nome de **campo**.

A cada linha da tabela (conjunto de campos) damos o nome de **registro**.

É importante lembrar que os dados são armazenados de maneira organizada nas tabelas e, algumas vezes, as informações isoladas de uma tabela podem parecer não fazer sentido. Porém, quando esses dados são relacionados a outros dados de outras tabelas, ocorre uma complementação dos dados.

Ao dado ou conjunto de dados interpretado damos o nome de **informação**. Ou seja, os dados passam a se chamar informações a partir do momento que sabemos para que servem, a partir do momento que passam a ter utilidade.

EXEMPLO DE TABELA DO SISTEMA WINTHOR



1.2.3 Relacionamento entre Tabelas

Podemos chamar de relacionamento entre tabelas a maneira que as tabelas utilizam para se 'ligarem', para que seus dados possam ser unidos, formando informações completas.

Os tipos de relacionamentos mais utilizados são:

- Um para Um (1:1)
- Um para Vários (1:N)

a) Relacionamento do Tipo Um para Um (1:1)

Esta relação existe quando os campos que se relacionam são ambos do tipo Chave Primária, em suas respectivas tabelas. Cada um dos campos não apresenta valores repetidos. Na prática existem poucas situações onde utilizaremos um relacionamento deste tipo.

Um exemplo típico desse tipo de relacionamento pode ser demonstrado com as tabelas PCPEDC (Cabeçalho de pedido de venda) e PCNFSAID (Nota fiscal de venda). Nesse exemplo, para cada **um** cabeçalho de pedido de venda existe **apenas uma** nota fiscal de venda.

b) Relacionamento do Tipo Um para Vários (1:N)

Este é, com certeza, o tipo de relacionamento mais comum entre duas tabelas. Uma das tabelas (o lado **um** do relacionamento) possui um campo que é a **Chave Primária** e a outra tabela (o lado **vários**) se relaciona através de um campo cujos valores relacionados **podem se repetir várias vezes**.

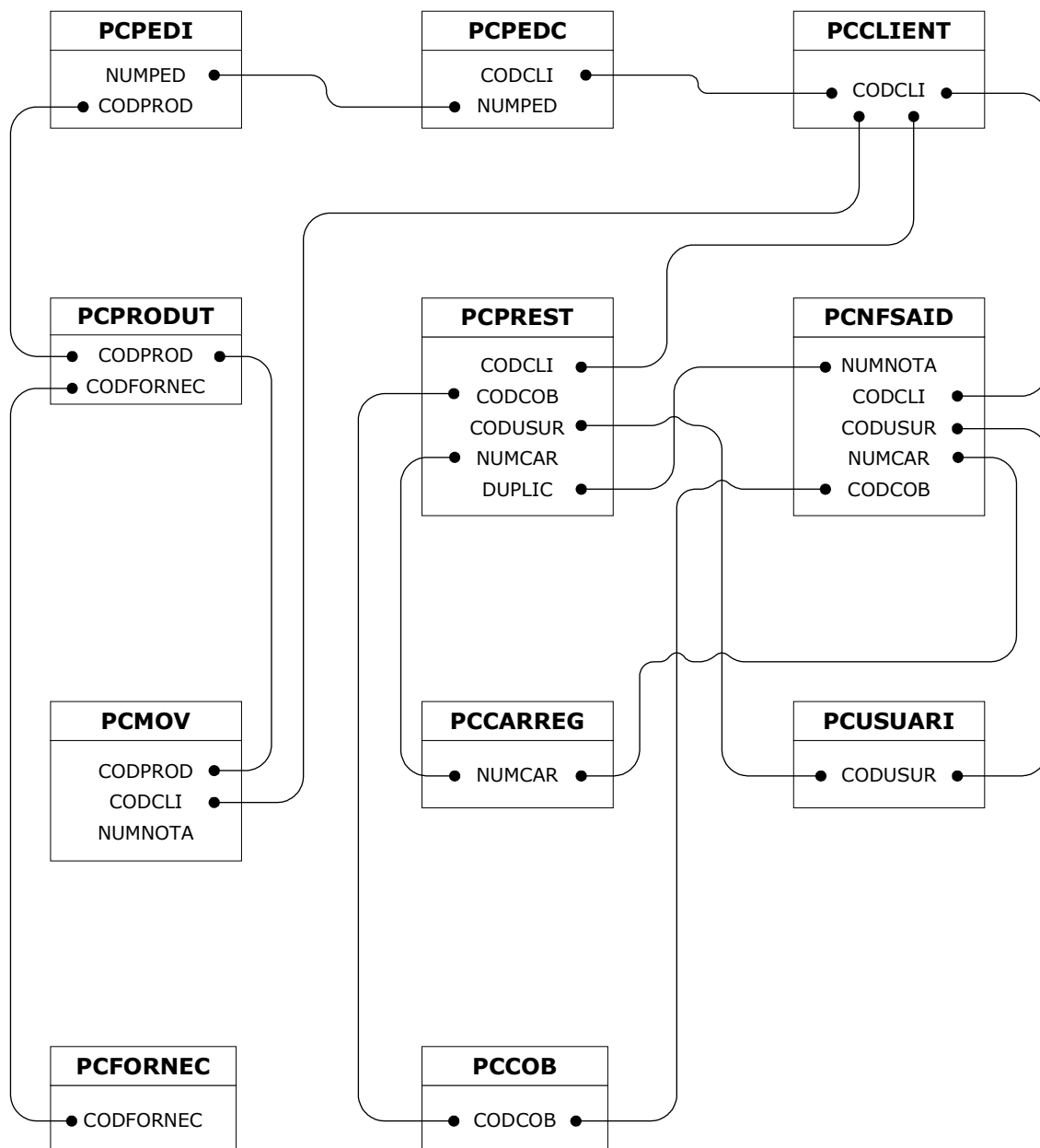
Um exemplo típico desse tipo de relacionamento pode ser demonstrado com as tabelas PCPEDC (Cabeçalho de pedido de venda) e PCPEDI (Itens de pedido de venda). Nesse exemplo, para cada **um** cabeçalho de pedido de venda podemos ter **vários** itens do mesmo pedido de venda.

2 Tabelas do Sistema Winthor

2.1 Principais Tabelas do Winthor

Nome da Tabela	Descrição da Tabela
PCCLIENT	Cadastro de Clientes
PCUSUARI	Cadastro de RCA's
PCEMPR	Cadastro de Funcionários
PCFORNEC	Cadastro de Fornecedores
PCPRODUT	Cadastro de Produtos
PCCOB	Cadastro de Cobranças
PCBANCO	Cadastro de Caixas / Bancos
PCCARREG	Tabela de Carregamentos
PCCONSUM	Tabela de Parâmetros do Winthor
PCCRECLI	Tabela de Créditos de Clientes
PCEST	Tabela de Estoque
PCFILIAL	Cadastro de Filiais
PCLANC	Tabela de Lançamentos de Contas a Pagar
PCMOEDA	Cadastro de Moedas
PCMOV	Tabela de Movimentação de Produtos
PCMOVCR	Tabela de Movimentação de Numerários
PCNFENT	Cabeçalho de Nota Fiscal de Entrada
PCNFSAID	Cabeçalho de Nota Fiscal de Saída
PCPEDC	Cabeçalho de Pedido de Venda
PCPEDI	Itens de Pedido de Venda
PCPLPAG	Cadastro de Planos de Pagamento
PCPRACA	Cadastro de Praças
PCPREST	Tabela de Contas a Receber
PCREGIAO	Cadastro de Regiões
PCROTA	Cadastro de Rotas
PCSUPERV	Cadastro de Supervisores
PCTABPR	Tabela de Preços

2.2 Relacionamento entre as tabelas do Winthor



3 Linguagem SQL

3.1 O que é linguagem SQL? Qual sua finalidade?

Structured Query Language, ou **Linguagem de Consulta Estruturada** ou **SQL**, é uma linguagem de pesquisa banco de dados.

O SQL foi desenvolvido originalmente no início dos anos 70 nos laboratórios da IBM. O nome original da linguagem era **SEQUEL**, acrônimo para "**Structured English Query Language**" (**Linguagem de Consulta Estruturada em Inglês**).

A linguagem SQL é um grande padrão de banco de dados. Isto decorre da sua simplicidade e facilidade de uso. Ela se diferencia de outras linguagens de consulta a banco de dados no sentido em que uma consulta SQL especifica a forma do resultado e não o caminho para chegar a ele. Ela é uma linguagem declarativa em oposição a outras linguagens procedurais. Isto reduz o ciclo de aprendizado daqueles que se iniciam na linguagem.

Embora o SQL tenha sido originalmente criado pela IBM, rapidamente surgiram vários "dialetos" desenvolvidos por outros produtores. Essa expansão levou à necessidade de ser criado e adaptado um padrão para a linguagem. Esta tarefa foi realizada pela American National Standards Institute (ANSI) em 1986 e ISO em 1987.

Tal como dito anteriormente, o SQL, embora padronizado pela ANSI e ISO, possui muitas variações e extensões produzidos pelos diferentes fabricantes de sistemas gerenciadores de bases de dados. Tipicamente a linguagem pode ser migrada de plataforma para plataforma sem mudanças estruturais.

3.2 Comando Select

O comando select é o 'comando base' para todos os comandos de pesquisa de informações no banco de dados. Este comando possui a seguinte estrutura padrão:

SELECT	(Selecionar)
<CAMPOS ou *>	(Campos selecionados ou todos os campos usando "*")
FROM <TABELAS>	(Das Tabelas)
WHERE <CLÁUSULAS>	(Cláusulas de amarrações e/ou restrições)
GROUP BY <CAMPOS>	(Agrupando por campos)
ORDER BY <CAMPOS>	(Ordenando por campos crescentes ou Decrescentes).

a. Selecionando todas as colunas ou colunas específicas.

```
select * from pcnfsaid;
```

(seleciona todos os campos da tabela pcnfsaid)

```
select numnota, dtsaida, vltotal from pcnfsaid;
```

(seleciona os campos numnota, dtsaida e vltotal da tabela pcnfsaid)

b. Usando operadores aritméticos (usando ou não parênteses)

```
select numnota, dtsaida, vltotal, vl desconto, vltotal-vl desconto from pcnfsaid;
```

(seleciona os campos numnota, dtsaida, vltotal, vl desconto, calcula vltotal-vl desconto, colhendo informações da tabela pcnfsaid)

c. Concatenar

O comando concatenar consiste em unificar as informações de dois campos distintos da tabela utilizada, de forma que sejam exibidas em apenas um campo.

```
select especie || serie, numnota from pcnfsaid;
```

(seleciona o campo especie, concatenando com o campo serie, seleciona também o campo numnota, todos da tabela pcnfsaid)

d. Distinct

O comando 'distinct' consiste em exibir as informações da tabela utilizada de maneira distinta, de forma que informações não sejam exibidas de maneira redundante.

```
select distinct codcli from pcnfsaid;
```

(seleciona o campo codcli da tabela pcnfsaid de maneira distinta, ou seja, cada informação do campo codcli será exibido apenas uma vez, não importando quantas outras vezes essa informação esteja presente em outros registros da tabela utilizada)

3.3 Restringindo Dados

(where)

O comando de restrição 'where' é o comando de restrição mais utilizado. O comando 'where' quer dizer 'onde', ou seja, com a utilização do comando 'where', serão exibidas as informações solicitadas, onde as mesmas obedecerem as restrições informadas.

a) **Condições de Comparação (=, >, >=, <, <=, <>, in, not in, like, not like, between, not between, is null, is not null)**

'=' → igual
'>' → maior
'>=' → maior ou igual
'<' → menor
'<=' → menor ou igual
'<>' → diferente
'in' → na lista
'not in' → não está na lista
'like' → como, parecido
'not like' → não é parecido
'between' → entre dois valores
'not between' → não está entre dois valores
'is null' → é nulo
'is not null' → não é nulo

Obs.: Campo nulo é o campo que não possui nenhum valor atribuído. Campos com um caractere de espaço ou 0(zero) tem valor atribuído. Campo nulo é campo vazio. Pode-se usar o comando TRIM para retirar os espaços do campo, como por exemplo: `select * from pcpedc where trim(condvenda) is null`

`select * from pcnfsaid where numnota = 50;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for igual a 50)

`Select * from pcnfsaid where numnota > 45;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for maior que 45)

`Select * from pcnfsaid where numnota >=50;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for maior ou igual a 50)

`Select * from pcnfsaid where numnota < 10;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for menor que 10)

`Select * from pcnfsaid Where numnota <= 5;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for menor ou igual a 5)

`Select * from pcnfsaid Where numnota <> 2;`
(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota for diferente de 2)

`Select * from pcnfsaid Where numnota in (1,3,5,6,8);`

(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota pertencer à lista 1,3,5,6,8)

*Select * from pcnfsaid Where numnota not in (1,3,5,6,8);*

(seleciona todos os campos da tabela pcnfsaid onde o valor do campo numnota não pertencer à lista 1,3,5,6,8)

*Select * from pcnfsaid Where numnota like '1%';*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota começar com o número 1, não importando quantos números venham na seqüência)

*Select * from pcnfsaid Where numnota like '1_';*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota começar com o número 1, e que tenham apenas um número na seqüência)

*Select * from pcnfsaid Where numnota not like '1%';*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota não começar com o número 1, não importando quantos números venham na seqüência)

*Select * from pcnfsaid Where numnota not like '1_';*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota não começar com o número 1, e que tenham apenas um número na seqüência)

*Select * from pcnfsaid Where numnota between 1 and 10;*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota estiver entre 1 e 10)

*Select * from pcnfsaid Where numnota not between 1 and 10;*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo numnota não estiver entre 1 e 10)

*select * from pcnfsaid where comissao is null;*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo comissao for nulo)

*select * from pcnfsaid where comissao is not null;*

(seleciona todos os campos da tabela pcnfsaid, onde o valor do campo comissao não for nulo)

b) Condições Lógicas (and, or)

A condição lógica '**and**' é utilizada para acrescentar uma condição de inclusão, ou seja, o resultado será exibido apenas se todos os critérios de pesquisa foram obedecidos.

A condição lógica '**or**' é utilizada para acrescentar uma condição de alternativa, ou seja, o resultado será exibido caso seja obedecido o primeiro critério OU caso seja obedecido o segundo critério.

```
select * from pcprest
where duplic = 2
and   codcob <> 'DESD';
```

(seleciona todos os campos da tabela pcprest, onde o campo duplic for igual a 2 e o campo codcob for diferente de DESD)

```
select * from pcprest
where duplic = 2
and   codcob = 'BK'
```



*or duplic = 3
and codcob = 'CHP';*

(seleciona todos os campos da tabela pcprest, onde o campo duplic for igual a 2, e o campo codcob for igual a BK, ou onde o campo duplic for igual a 3, e o campo codcob for igual a CHP)

3.4 Regras de Precedência

- ✓ Operadores aritméticos
- ✓ Operador de concatenação
- ✓ Condições de comparação
- ✓ Is null, is not null, like, not in
- ✓ Between, not between
- ✓ Condição lógica and
- ✓ Condição lógica or

Obs1.: As regras de precedência podem ser alteradas utilizando-se parênteses.

Obs2.: Recomenda-se sempre utilizar parênteses, para melhor organização e visualização do código escrito.

3.4.1 Ordenando os resultados

(order by, order by desc)

a) Ordenando por coluna simples

```
select * from pcempr  
order by matricula;
```

(seleciona todos os campos da tabela pcempr, ordenando as informações pelo campo matricula, em ordem crescente)

```
select * from pcempr  
order by matricula desc;
```

(seleciona todos os campos da tabela pcempr, ordenando as informações pelo campo matricula, em ordem decrescente)

b) Ordenando por múltiplas colunas

```
select * from pcprest  
order by duplic, codcli, codcob;
```

(seleciona todos os campos da tabela pcprest, ordenando as informações pelos campos duplic, codcli e codcob, em ordem crescente)

Obs.: O termo 'desc' é aplicado ao campo diretamente anterior ao termo, sendo aplicado de maneira individual. Caso se deseje aplicar ordem decrescente à todos os campos, o termo 'desc' deve ser obrigatoriamente informado após cada campo.

3.5 Funções de Linha Simples

3.5.1 Funções gerais

(NVL, CASE, DECODE)

select nvl(punit,0) from pcmov

(seleciona o campo punit da tabela pcmov, atribuindo valor 0 se for nulo)

*select numnota, case when numnota < 100 then 'MENOR QUE 100'
when numnota between 101 and 200 then 'ENTRE 101 E 200'
else 'MAIOR QUE 200' end "RESULTADO"*

from pcmov;

(seleciona da tabela pcmov o campo numnota, e exibe mensagem de acordo com as condições: caso o campo numnota seja menor que 100, exibe a mensagem 'MENOR QUE 100', caso o campo numnota esteja entre 101 e 200, exibe a mensagem 'ENTRE 101 E 200', caso contrário, exibe a mensagem 'MAIOR QUE 200')

select numnota, decode (numnota, 100, 'CEM', 200, 'DUZENTOS', 300, 'TREZENTOS', 'NENHUM') resultado from pcmov;

(seleciona da tabela pcmov, o campo numnota, e exibe mensagem de acordo com as condições: se o campo numnota for igual a 100, exibe 'CEM', se for 200, exibe 'DUZENTOS', se for 300, exibe 'TREZENTOS', se não for nenhum dos anteriores, exibe 'NENHUM')

3.5.2 Funções de Caractere

(LOWER, UPPER, INITCAP, SUBSTR, LENGTH, REPLACE, INSTR)

select matricula, lower(nome) from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, convertendo o conteúdo do campo nome para que seja exibido todo em caracteres minúsculos)

select matricula, upper(nome) from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, convertendo o conteúdo do campo nome para que seja exibido todo em caracteres maiúsculos)

select matricula, initcap(nome) from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, convertendo o conteúdo do campo nome para que seja exibido em maiúsculo toda primeira letra de cada palavra)

select matricula, substr(nome,1,5) from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, exibindo, do campo nome, apenas o conteúdo a partir do primeiro caractere, os próximos 5 caracteres)

select matricula, nome, length(nome) from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, e exibe, logo em seguida, o tamanho preenchido do campo nome)

select matricula, nome, replace(nome,'A','') from pccmpr;*

(seleciona os campos matricula e nome da tabela pccmpr, e exibe, logo em seguida, o campo nome, substituindo o caractere 'A' por '')*

select matricula, nome, instr(nome,'A') from pccmpr;

(seleciona os campos matricula e nome da tabela pccmpr, e exibe, logo em seguida, a posição do primeiro caractere 'A' encontrado no conteúdo do campo nome. Caso o caractere desejado não existir, a função retornará valor 0.)

3.5.3 Funções Numéricas

(ROUND, TRUNC, MOD, ABS)

*select codprod, numregiao, ptabela, round(ptabela,2) from pctabpr;
(seleciona os campos codprod, numregiao, ptabela da tabela pctabpr, e exibe, logo em seguida, o campo ptabela com seu valor arredondado para 2 casas decimais)*

*select codprod, numregiao, ptabela, trunc(ptabela,2) from pctabpr;
(seleciona os campos codprod, numregiao, ptabela da tabela pctabpr, e exibe, logo em seguida, o campo ptabela com seu valor truncado na segunda casa decimal).*

*select mod(300,200) from dual;
(seleciona o resto da divisão entre 300 e 200).*

*select abs(perdesc) from pcpedi;
(seleciona o valor absoluto do campo perdesc da tabela pcpedi).
Obs.: Valor absoluto é o 'valor puro' sem o sinal que determina se o mesmo é positivo ou negativo.*

3.6 Trabalhando com Datas

3.6.1 Função SYSDATE

select sysdate from dual;
(seleciona a data do sistema)

3.6.2 Aritmética com Datas

a) Adicionando dias

select sysdate+2 from dual;
(seleciona a data do sistema adicionando 2 dias)

b) Subtraindo dias

select sysdate-2 from dual;
(seleciona a data do sistema subtraindo 2 dias)

c) Diferença entre datas

select numnota, dtsaida, dtentrega, dtentrega-dtsaida diferenca from pcnfsaid;
(seleciona os campos numnota, dtsaida, dtentrega da tabela pcnfsaid, e exibe, logo em seguida, a diferença entre os campos dtentrega e dtsaida, apelidando o campo com o nome de 'diferenca')

3.7 Funções de Conversão

a) NUMBER to VARCHAR

*select dtmov, numnota, to_char(numnota,'000000') from pcmov;
(seleciona os campos dtmov e numnota da tabela pcmov, e, logo em seguida, o campo numnota sendo convertido para caracter, no formato de 6 dígitos)*

b) DATE to VARCHAR

*select dtmov, numnota, to_char(dtmov,'dd/mm') from pcmov;
(seleciona os campos dtmov e numnota da tabela pcmov, e, logo em seguida, o campo dtmov sendo convertido para caracter, no formato 'dd/mm')*

3.8 Funções de Grupo

(AVG, COUNT, MAX, MIN, SUM)

```
select avg(vltotal) from pcnfsaid;  
(seleciona a média dos valores do campo vltotal da tabela pcnfsaid)
```

```
select count(numnota) from pcnfsaid;  
(efetua contagem do campo numnota da tabela pcnfsaid)
```

```
select max(vltotal) from pcnfsaid;  
(seleciona o maior valor do campo vltotal da tabela pcnfsaid)
```

```
select min(vltotal) from pcnfsaid;  
(seleciona o menor valor do campo vltotal da tabela pcnfsaid)
```

```
select sum(vltotal) from pcnfsaid;  
(seleciona a soma dos valores do campo vltotal da tabela pcnfsaid)
```

3.8.1 GROUP BY

```
select dtmov, codprod, sum(qt)  
from pcmov  
where codprod in (0,1)  
group by codprod, dtmov;  
(seleciona os campos dtmov, codprod, a soma do campo qt, da tabela pcmov, onde o  
campo codprod for 0 ou 1, agrupando as informações pelos campos codprod e dtmov)
```

HAVING

O termo 'having' tem funcionalidade igual ao termo 'where', porém, é destinado exclusivamente para verificações em resultados de funções de grupo.

```
Select codcli, count(numnota)  
From pcnfsaid  
Group by codcli  
Having count(numnota) > 100  
(seleciona o campo codcli e efetua a contagem das notas fiscais deste cliente, exibindo  
apenas os clientes que tiverem quantidade de notas fiscais superior a 100).
```

3.9 Relacionamento entre tabelas

```
select a.codcli, b.cliente, a.numnota
from pcnfsaid a, pcclient b
where a.codcli = b.codcli;
```

(seleciona os campos codcli[pcnfsaid], cliente[pcclient] e numnota[pcnfsaid], das tabelas pcnfsaid e pcclient, onde o valor do campo codcli da tabela pcnfsaid for igual ao valor do campo codcli da tabela pcclient. Os resultados serão exibidos apenas se essa condição for obedecida)

```
select a.codcli, b.cliente, a.numnota
from pcnfsaid a, pcclient b
where a.codcli = b.codcli (+);
```

(seleciona os campos codcli[pcnfsaid], cliente[pcclient] e numnota[pcnfsaid], das tabelas pcnfsaid e pcclient, onde o valor do campo codcli da tabela pcnfsaid for igual ao valor do campo codcli da tabela pcclient. Caso os valores dos campos não sejam iguais, as informações solicitadas da tabela pcnfsaid serão exibidas, e os campos da tabela pcclient serão exibidos em branco)

3.9.1 Sub-select's

```
select * from pcprest where codusur in (select distinct(codusur) from pcusuari);
```

(seleciona todos os campos da tabela pcprest, onde o valor do campo codusur pertencer à seleção distinta do campo codusur da tabela pcusuari)

Union / union all

O comando union tem como função unir o resultado de 2 ou mais script's distintos e independentes. A única obrigatoriedade do comando union é que todos os script's envolvidos devem ter exatamente a mesma quantidade de campos, e exatamente com os mesmos apelidos.

O comando union irá unir como resultado final apenas as linhas de resultado que sejam únicas em cada script, ou seja, não ocorrerá a existência de linhas repetidas.

O comando union all, por sua vez, irá unir como resultado final todas as linhas de resultado dos script's, sejam essas linhas repetidas ou não entre os script's utilizados.

Toda vez que se utilizar o comando union ou union all, a quantidade de campos e seus nomes devem ser exatamente os mesmos em todas as consultas envolvidas.

```
Select numnota, codcli
From pcnfsaid
```

Union

```
Select numnota, codcli
From pcnfent
```

(seleciona a número de nota e código do cliente da tabela PCNFSaid, unindo com o número de nota e código de cliente da tabela PCNFENT. Serão considerados na união apenas os registros não repetidos nas pesquisas.)

```
Select numnota, codcli  
From pcnf Said
```

```
Union all
```

```
Select numnota, codcli  
From pcnf Fent
```

(seleciona a número de nota e código do cliente da tabela PCNFSAID, unindo com o número de nota e código de cliente da tabela PCNFENT. Serão considerados na união todos os registros das duas pesquisas, sejam esses registros repetidos ou não.)

Sub-select no 'from'

A utilização de sub-select's na cláusula 'from' permite que script's independentes sejam utilizados como tabelas, e que possa ser estabelecido relacionamento entre esses script's independentes.

```
Select compras.codprod, compras.qtde as "Compras", vendas.qtde as "Vendas"  
From (select codprod codprod, sum(qt) qtde from pcmov where codoper = 'E') as Compras,  
(select codprod codprod, sum(qt) qtde from pcmov where codoper = 'S') as Vendas  
where compras.codprod = vendas.codprod;
```

3.10 Exercícios

1) Montar pesquisa com as seguintes informações:

- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Código do primeiro RCA
- ✓ Nome do primeiro RCA

Observações:

- ✓ Tabelas utilizadas (PCCLIENT / PCUSUARI)

2) Montar pesquisa com as seguintes informações:

- ✓ Espécie da N.F.
- ✓ Número da N.F.
- ✓ Data de saída da N.F.
- ✓ Valor Total da N.F.
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Código do RCA
- ✓ Nome do RCA
- ✓ Código do Supervisor
- ✓ Nome do Supervisor

Observações:

- ✓ Tabelas utilizadas (PCNFSAID / PCCLIENT / PCUSUARI / PCSUPERV)
- ✓ Período solicitado: de 01/07/2005 a 31/07/2005

3) Montar pesquisa com as seguintes informações:

- ✓ Data de movimentação do produto
- ✓ Número da N.F.
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Código do produto
- ✓ Descrição do produto
- ✓ Código de operação da movimentação
- ✓ Quantidade movimentada
- ✓ Preço unitário do produto
- ✓ Valor total da movimentação (quantidade movimentada * preço unitário do produto)

Observações:

- ✓ Tabelas utilizadas (PCMOV / PCCLIENT / PCPRODUT)
- ✓ Período solicitado: de 01/08/2005 a 31/08/2005
- ✓ Código de operação = 'S'

4) Montar pesquisa com as seguintes informações:

- ✓ Número da nota de venda
- ✓ Data da nota de venda
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Código do RCA
- ✓ Nome do RCA
- ✓ Número da prestação
- ✓ Data de vencimento
- ✓ Valor
- ✓ Código de cobrança
- ✓ Data de pagamento
- ✓ Valor pago

Observações:

- ✓ Tabelas utilizadas (PCPREST / PCNFSAID / PCCLIENT / PCUSUARI)
- ✓ Mostras informações do período de 01/08/2005 a 31/08/2005 das prestações que não foram desdobradas (diferente de DESD)

5) Montar pesquisa com as seguintes informações:

- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ CGC ou CPF do cliente (do jeito que estiver na tabela)
- ✓ CGC ou CPF do cliente (sem máscara)
- ✓ CGC ou CPF do cliente com máscara correta.

Observações:

- ✓ Tabelas utilizadas (PCCLIENT)
- ✓ Se for CGC (14 dígitos), colocar máscara '99.999.999/9999-99'
- ✓ Se for CPF (11 dígitos), colocar máscara '999.999.999-99'
- ✓ Se não for CGC nem CPF informar 'TIPO INCORRETO'
- ✓ Ordenado por código de cliente

6) Montar pesquisa com as seguintes informações:

- ✓ Numero da Nota
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Cidade do cliente
- ✓ Código do primeiro RCA
- ✓ Nome do primeiro RCA
- ✓ Campo descritivo do tipo de RCA (Externo, Interno, Representante)
- ✓ Valor da nota
- ✓ Percentual de comissão da nota
- ✓ Valor da comissão do RCA

Observações:

- ✓ Tabelas utilizadas (PCNFSAID / PCCLIENT / PCUSUARI)
- ✓ Filtrar pelo período de 01/01/2005 a 30/06/2005

7) Montar pesquisa com as seguintes informações:

- ✓ Código da praça do pedido de venda
- ✓ Nome da praça
- ✓ Número do pedido de venda
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Código de cobrança do pedido
- ✓ Descrição da cobrança
- ✓ Código do plano de pagamento
- ✓ Descrição do plano de pagamento
- ✓ Prazo médio do plano de pagamento.

Observações:

- ✓ Tabelas utilizadas (PCPEDC / PCPRACA / PCCLIENT / PCCOB / PCPLPAG)
- ✓ O cliente não precisa necessariamente estar cadastrado
- ✓ Apenas do supervisor 1
- ✓ Apenas dados da filial 1
- ✓ Período de 01/01/2006 a 30/04/2006
- ✓ Apenas vendas da cobrança 237

8) Montar pesquisa com as seguintes informações:

- ✓ Texto fixo informando que a nota fiscal é de venda (saída)
- ✓ Número da nota fiscal
- ✓ Data de emissão da nota fiscal
- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Valor da nota fiscal

Unindo com

- ✓ Texto fixo informando que a nota fiscal é de entrada
- ✓ Número da nota fiscal
- ✓ Data de emissão da nota fiscal
- ✓ Código do fornecedor
- ✓ Nome do fornecedor
- ✓ Valor da nota fiscal

Observações:

- ✓ Tabelas utilizadas
 - Pesquisa 1: (PCNFSAID / PCCLIENT)
 - Pesquisa 2: (PCNFENT / PCFORNEC)
- ✓ Período de 01/01/2006 a 30/04/2006

9) Montar pesquisa com as seguintes informações:

- ✓ Código do cliente
- ✓ Nome do cliente
- ✓ Valor acumulado de vendas
- ✓ Valor acumulado de devoluções

Observações:

- ✓ Script deve utilizar sub-select no from
- ✓ Dados de vendas e devoluções não são obrigatórios
- ✓ Tabelas utilizadas
 - Pesquisa principal: PCCLIENT
 - Pesquisa de vendas: PCNFSAID
 - Pesquisa de devoluções: PCNFENT
- ✓ Período de vendas e devoluções: 01/01/2006 a 30/04/2006

4 Gabarito dos Exercícios

4.1 Linguagem SQL

- 1) Código do cliente, nome do cliente, código do primeiro RCA, nome do primeiro RCA. (Tabelas: PCCLIENT / PCUSUARI)

```
select c.codcli, c.cliente, u.codusur, u.nome
from pcclient c, pcusuari u
where c.codusur1 = u.codusur;
```

-
- 2) Espécie da N.F., número da N.F., data de saída da N.F., código do cliente, nome do cliente, código do RCA, nome do RCA, código do supervisor, nome do supervisor. (Tabelas: PCNFSAID / PCCLIENT / PCUSUARI / PCSUPERV)
 - ✓ Período solicitado: de 01/07/2005 a 31/07/2005

```
select n.especie, n.numnota, n.dtsaida, n.vltotal, n.codcli, c.cliente,
       n.codusur, u.nome, n.codsupervisor, u.nome
from pcnfsaid n, pcclient c, pcusuari u, pcsuperv s
where n.codcli = c.codcli
and n.codusur = u.codusur
and n.codsupervisor = s.codsupervisor
and dtsaida between '01-jul-2005' and '31-jul-2005';
```

-
- 3) Data de movimentação do produto, número da N.F., código do cliente, nome do cliente, código do produto, descrição do produto, código de operação da movimentação, quantidade movimentada, preço unitário do produto, valor total da movimentação (quantidade movimentada * preço unitário do produto). (Tabelas: PCMOV / PCCLIENT / PCPRODUT)
 - ✓ Período solicitado: de 01/08/2005 a 31/08/2005
 - ✓ Código de operação = 'S'

```
select m.dtmov, m.numnota, m.codcli, c.cliente, m.codprod, p.descricao,
       m.codoper, m.qt, m.punit, (m.qt * m.punit) vltotal
from pcmov m, pcclient c, pcprodut p
where m.codcli = c.codcli
and m.codprod = p.codprod
and dtmov between '01-aug-2005' and '31-aug-2005'
and codoper = 'S'
order by m.dtmov, m.numnota;
```

- 4) Número de nota de venda, data da nota de venda, código do cliente, nome do cliente, código do RCA, nome do RCA, prestação, data de vencimento, valor, código de cobrança, data de pagamento e valor pago. (Tabelas: PCNFSaid, PCCLIENT, PCUSUARI, PCPREST)
- ✓ Mostra informações do período de 01/08/2005 a 31/08/2005 das prestações que não foram desdobradas (diferente de DESD)

```
select a.numnota, a.dtsaida, a.codcli, b.cliente, a.codusur, c.nome,
       d.prest, d.dtvinc, d.valor, d.codcob, d.dtpag, vpago
from pcnfsaid a, pcclient b, pcusuari c, pcprest d
where a.codcli = b.codcli
and a.codusur = c.codusur
and d.duplic = a.numnota
and d.codcli = a.codcli
and d.codcob <> 'DESD'
and a.dtsaida between '01-aug-2005' and '31-aug-2005'
order by a.dtsaida, a.numnota, a.codcli, d.prest;
```

- 5) Trazer código do cliente, nome do cliente, cgc ou cpf do cliente(do jeito que estiver na tabela), cgc ou cpf do cliente(sem máscara), cgc ou cpf do cliente com máscara correta. (Tabelas: PCCLIENT)
- ✓ Se for CGC (14 dígitos), colocar máscara '99.999.999/9999-99'
- ✓ Se for CPF (11 dígitos), colocar máscara '999.999.999-99'
- ✓ Se não for CGCnem CPF informat 'TIPO INCORRETO'
- ✓ Ordenado por código de cliente

```
select codcli, cliente, cgcent,
       replace(replace(replace(cgcent,'.', ''), '-', ''), '/', '') CGC,
       case when length (replace(replace(replace(cgcent,'.', ''), '-', ''), '/', ''))=14 then
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),1,2) || '.' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),3,3) || '.' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),6,3) || '/' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),9,4) || '-' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),13,2)
       when length (replace(replace(replace(cgcent,'.', ''), '-', ''), '/', ''))=11 then
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),1,3) || '.' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),4,3) || '.' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),7,3) || '-' ||
substr(RPAD(REPLACE(REPLACE(REPLACE(NVL(PCCLIENT.CGCEM,'0'),'.', ''), '-', ''), '/', ''),18
),10,2)
       else 'TIPO INCORRETO' end tipo
from pcclient
order by codcli;
```

- 6) Número da nota, código de cliente, nome do cliente, cidade do cliente, código do primeiro RCA, nome do primeiro RCA, campo descritivo do tipo do RCA, valor da nota,

percentual de comissão da nota, valor da comissão do RCA. (Tabelas utilizadas PCNFSaid, PCCLIENT, PCUSUARI)

- ✓ Filtrar pelo período de 01/01/2005 a 30/06/2005

```
SELECT n.numnota, n.codcli, c.cliente, c.municent, n.codusur, u.nome,
       DECODE(u.tipovend, 'R', 'REPRESENTANTE', 'I', 'INTERNO', 'E', 'EXTERNO',
       'NAO INFORMADO'), N.vltotger,
       ROUND(((N.comissao/N.vltotger)*100),2) AS PERC_COMISS, N.comissao
FROM pcnfsaid n, pcclient c, pcusuari u
WHERE N.codcli=c.codcli
      AND n.codusur=u.codusur
      AND n.vltotger>0
      AND n.dtsaida BETWEEN '01-jan-2005' AND '30-jun-2005';
```

- 7) Código da praça do pedido, nome da praça, numero do pedido, código do cliente, nome do cliente, código da cobrança do pedido, descrição da cobrança, código do plano de pagamento, descrição do plano de pagamento, prazo médio do plano de pagamento. (Tabelas utilizadas: PCPEDC, PCPRACA, PCCLIENT, PCCOB, PCPLPAG).

- ✓ O cliente não precisa necessariamente estar cadastrado
- ✓ Apenas vendas tipo 1
- ✓ Apenas do supervisor 1
- ✓ Apenas dados da filial 1
- ✓ Período de 01/01/2006 a 30/04/2006
- ✓ Apenas vendas da cobrança 237

```
SELECT p.codpraca, r.praca, p.numped, p.codcli, c.cliente, p.codcob, b.cobranca,
p.codplpag, g.descricao, g.numdias
FROM PCPEDC p, PCPRACA r, PCCLIENT c, PCCOB b, PCPLPAG g
WHERE p.codpraca=r.codpraca
      AND p.codcli=c.codcli (+)
      AND p.codcob=b.codcob
      AND p.codplpag=g.codplpag
      AND p.condvenda=1
      AND p.codsUPERVISOR='1'
      AND p.codfilial='1'
      AND p.data BETWEEN '01-jan-2006' AND '30-apr-2006'
      AND p.codcob='237';
```

1) Mais ou menos

```
select      nfs.numnota,      nfs.dtsaida,      cli.codcli,      cli.cliente,      nfs.vltotal
from        pcnfsaid          nfs,              pcclient          cli
where       nfs.codcli        =                  cli.codcli
union
select      e.numnota,      e.dtemissao,      f.codfornec,      f.fornecedor,      e.vltotal
from        pcnfent          e,                  pfornec          f
where       e.codfornec = f.codfornec
```

2) Mais ou menos

```
select      compras.codprod,      compras.qtde      as      "compras",      vendas.qtde      as      "vendas"
from
```



Nossos clientes recomendam.

```
(select codprod codprod, sum (qt) qtde from pcmov where codoper like 'E%' group by codprod)
compras,
(select codprod codprod, sum(qt) qtde from pcmov where codoper like 'S%' group by
codprod)Vendas
where compras.codprod = vendas.codprod;
```